CPEN6261010 · MICROCONTROLLER

PlatformIO, Git, Copilot, and Debugging

Native AVR C from source to evidence



SESSION 3

ATmega328P · bare-metal AVR C

Session 1 and 2 become a workflow

01

Flash stores instructions

02

Registers expose state

03

Clock advances state

04

Pins reveal behavior

Session 3 adds the tools and evidence that move a design through all four.

Source to silicon

01

main.c

02

preprocess and compile

03

object and link

04

ELF and HEX

05

program Flash

06

reset and run

Each stage produces different evidence and different failure messages.

Four checkpoints

BUILD	Can the toolchain produce ELF and HEX?
UPLOAD	Did bytes reach the intended device?
RUNTIME	Does the program logic behave as intended?
HARDWARE	Do pin mapping, wiring, power, and loads agree?

Development tool boundaries

VS Code	Editor and interface
PlatformIO	Project configuration and build orchestration
avr-gcc and avr-libc	Compiler, linker, headers, and C library
avrdude	Upload through bootloader or programmer
ATmega328P	Runtime hardware

Arduino board and Arduino framework

ARDUINO UNO BOARD

- ATmega328P target
- 16 MHz board clock
- USB to serial circuit
- bootloader and pin headers

ARDUINO FRAMEWORK

- Arduino.h
- setup and loop
- pinMode and digitalWrite
- C++ core and libraries

Today: use the board. Omit the framework.

PlatformIO project anatomy

```
avr-native-blink/
  platformio.ini
  src/
    main.c
  include/
  docs/
  tests/evidence/
  hardware/
```

PROJECT RULES

- src holds compiled C sources
- include holds project headers
- .pio is generated and ignored by Git
- docs, evidence, and hardware support the final project

Native AVR PlatformIO configuration

```
[env:uno_baremetal]
platform = atmelavr
board = uno
board_build.f_cpu = 16000000L
upload_protocol = arduino
upload_speed = 115200
build_flags =
    -Os -std=gnu11 -Wall -Wextra -Werror
```

The missing line matters

- No framework = arduino
- board selects ATmega328P and upload defaults
- upload_protocol names the bootloader protocol
- warnings become build failures for classroom code

Bare-metal main.c

```
#define F_CPU 16000000UL
#include <avr/io.h>
#include <util/delay.h>

int main(void)
{
    DDRB |= _BV(DDB5);
    for (;;)
    {
        PORTB ^= _BV(PORTB5);
        _delay_ms(500);
    }
}
```

Hardware meaning

- D13 maps to PB5
- DDB5 enables output drive
- PORTB5 changes the output latch
- F_CPU guides delay calculation

Build, clean, and upload commands

`pio run`	Compile and link the default environment
`pio run -t upload`	Build if needed, then program the board
`pio run -t clean`	Remove generated build output
`--upload-port COM5`	Use only when automatic port detection needs help

Build artifacts and their evidence

.pio/build/uno_baremetal

- src/main.o
- firmware.elf
- firmware.hex
- size report in terminal

WHAT EACH PROVES

- object source compiled
- linking and addresses resolved
- programming representation created
- Flash and SRAM estimate available

Demonstration 1 setup

Source-to-silicon blink

1. Locate the file
2. Run pio run and locate ELF plus HEX
3. Upload to the Uno
4. Observe D13 / PB5

EXPECTED

LED changes state every 500 ms. One full cycle takes about 1 s.

Demonstration 1 evidence

Build	SUCCESS, no warnings
Artifacts	firmware.elf and firmware.hex
Upload	avrdude completion
Runtime	D13 changes every 500 ms
Mapping	D13 equals PB5

This commit becomes the known-good baseline.

Failure stage changes the next action

Compiler error

Read the first source diagnostic

Linker error

Find missing or duplicate definitions

Upload error

Check port, cable, bootloader, and protocol

Runtime symptom

Measure code assumptions and hardware

Evidence-based debug loop



Changing several things at once destroys the evidence.

Runtime fault

The external D7 LED stays off

1. Wire LED and resistor to D7 / PD7
2. Upload fault_wrong_pin code
3. External D7 stays off
4. Onboard D13 blinks

EXPECTED

Build and upload succeed. Activity appears on a different physical pin.

Observations before edits

Evidence	Observed	What it rules in or out
Build	SUCCESS	Source and link stages completed
Upload	Completed	A binary reached the target
D13	Blinks	CPU, clock, loop, and timing are active
D7	Off	Expected pin path is not being driven

One hypothesis and one variable

HYPOTHESIS

- The program drives PB5 / D13
- The expected output is PD7 / D7
- The board and source names disagree

ONE-VARIABLE TEST

- Replace DDRB with DDRD
- Replace DDB5 with DDD7
- Replace PORTB5 with PORTD7
- Keep wiring and delay unchanged

This is one conceptual variable: the selected output pin.

AVR headers bridge names to hardware



The names are C preprocessor definitions grounded in the selected MCU.

Where avr-libc headers live on Windows

%USERPROFILE%\.platformio\packages\toolchain-atmelavr\avr\include

avr\io.h

avr\iom328p.h

avr\portpins.h

avr\sfr_defs.h

util\delay.h

If core_dir was customized, run pio system info and inspect the Core Directory first.

io.h selects the device header

```
#elif defined (__AVR_ATmega328P__)
#  include <avr/iom328p.h>
```

Where the symbol comes from

- PlatformIO invokes avr-gcc with -mmcu=atmega328p
- avr-gcc defines __AVR_ATmega328P__
- io.h includes one matching device header

iom328p.h declares registers and bits

```
#define PINB  _SFR_IO8(0x03)
#define PINB2 2

#define DDRB  _SFR_IO8(0x04)
#define DDB2  2

#define PORTB  _SFR_IO8(0x05)
#define PORTB2 2
```

Two kinds of definition

- PINB, DDRB, PORTB identify registers
- PINB2, DDB2, PORTB2 are bit positions
- All three bit names equal numeric position 2

sfr_defs.h creates a volatile byte

```
#define __SFR_OFFSET 0x20
#define _MMIO_BYTE(a) (*(volatile uint8_t *)(a))
#define _SFR_IO8(a) _MMIO_BYTE((a) + __SFR_OFFSET)
#define _BV(bit) (1 << (bit))
```

PORTB expansion

- I/O address 0x05
- plus offset 0x20
- data address 0x25
- volatile access to hardware

PB2, PORTB2, DDB2, and PINB2

NUMERIC RESULT

- PB2 = 2
- PORTB2 = 2
- DDB2 = 2
- PINB2 = 2

SEMANTIC ROLE

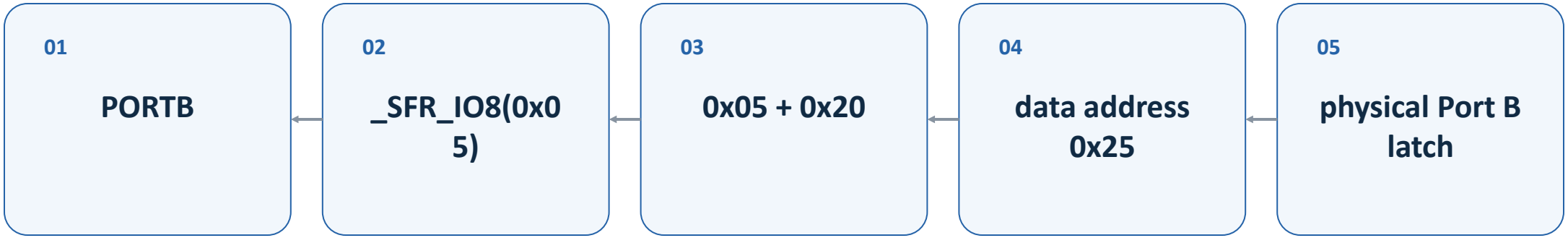
- generic Port B pin position
- output-latch bit position
- direction bit position
- input-sample bit position

portpins.h supplies aliases when a device header defines only one naming form.

Go to Definition in VS Code

1. Build the PlatformIO project once
2. Place the cursor on PORTB or DDB5
3. Ctrl+Click or press F12
4. Continue into _SFR_IO8 and _MMIO_BYTE

The Session 2 address map reappears



Header navigation confirms the address model rather than replacing it.

Header-file understanding check

1

Which file selects iom328p.h?

Answer: avr/io.h

2

What numeric value does DDB2 represent?

Answer: 2

3

What does _BV(DDB2) produce?

Answer: a mask with bit 2 set

4

Why is a register access volatile?

Answer: hardware can change or react outside ordinary variable rules

Two paths write Flash

BOOTLOADER UPLOAD

- USB to serial bridge
- bootloader already in Flash
- serial protocol after reset
- convenient for classroom iteration

ISP / ICSP PROGRAMMING

- external programmer
- SPI programming signals plus RESET
- works without application bootloader
- can program a blank device

Uno bootloader upload path



The uploader protocol can be Arduino-compatible while the application remains native AVR C.

ISP and the ICSP header

SIGNAL	ATmega328P	ROLE
MOSI	PB3	programmer to target
MISO	PB4	target to programmer
SCK	PB5	programming clock
RESET	PC6	enter programming mode
VCC	supply reference	confirm who powers target
GND	common reference	always connect

Programmer and debugger

PROGRAMMER

- erase and write memories
- read signatures and fuses
- verify programmed bytes
- does not normally halt source lines

DEBUGGER

- halt and resume execution
- step through code
- inspect registers and memory
- requires target support and a compatible probe

ATmega328P debugging boundary

The Uno has no onboard hardware debug probe.

- ATmega328P supports debugWIRE through RESET with compatible equipment
- Enabling debugWIRE changes configuration and can complicate ISP access
- This session uses evidence, measurements, and Git recovery

Do not change fuses live during the core class.

Tool choice check

Symptom or goal	First useful tool
Syntax error in main.c	Compiler diagnostic
Upload timeout	Port, cable, bootloader, or programmer check
Wrong pin toggles	Pin map, source diff, meter or analyzer
Blank new ATmega328P	ISP programmer
Inspect an earlier working version	Git switch to a detached commit

Git and GitHub are different layers

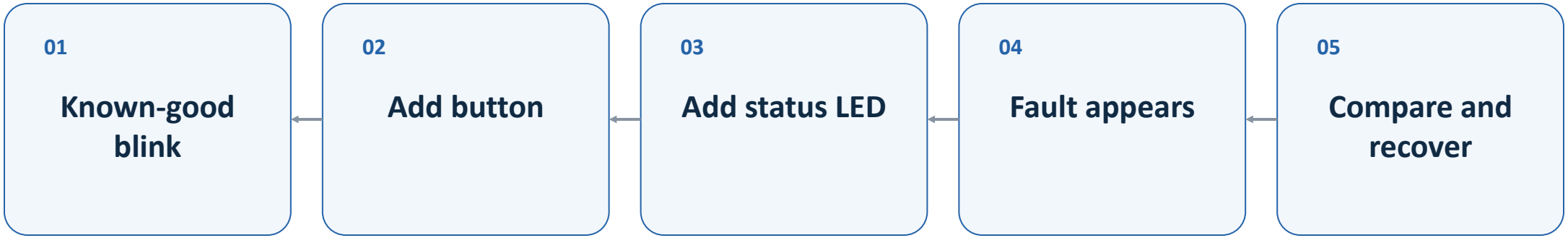
GIT

- local version history
- works offline
- commits, tags, diffs, restore, revert
- debug and recovery tool

GITHUB

- remote hosting service
- sharing and review
- team collaboration and backup
- also hosts Copilot as a separate service

Git history as debugging evidence



A useful commit says what changed and has a tested observation behind it.

Core known-good Git workflow

```
git init
git add .gitignore platformio.ini src/main.c
git diff --staged
git commit -m "feat: blink onboard LED"
git tag known-good-blink

git status
git diff
git log --oneline --decorate
```

Commit only after evidence

- clean build
- successful upload
- expected observation
- meaningful message

Restore, revert, and reset

Command	Use	History effect	Risk
git restore file	Discard an uncommitted file edit	No new commit	Loses that working change
git revert commit	Undo a committed change safely	Adds an inverse commit	Preserves shared history
git reset --hard commit	Move branch and match files exactly	Rewrites branch position	Can discard work and commits

Copilot is an untrusted assistant

A suggestion becomes engineering work only after verification.

- Inspect registers and assumptions
- Check datasheet and headers
- Build with warnings
- Test the real hardware
- Explain every accepted line

Plausible AI output with a runtime fault

```
int main(void)
{
    for (;;)
    {
        PORTB ^= _BV(PORTB5);
        _delay_ms(500);
    }
}
```

Why it is wrong

- PB5 never becomes an output
- PORTB toggles the input pull-up
- the code can compile cleanly
- hardware behavior reveals the missing DDR step

AI verification ladder

1. Inspect every include, register, bit, pin, and timing assumption
2. Open definitions and authoritative documentation
3. Build and read every warning
4. Upload and collect physical evidence
5. Modify, retest, explain, and record AI use

Final-project engineering habits

Datasheets

justify components and register behavior

Git history

show controlled contributions and recovery

Evidence

support results and debugging claims

Explainable code

meet originality and Q&A expectations

LCD

TODAY'S SESSION

- command versus character data
- RS and EN roles
- four-bit high nibble then low nibble
- initialization before text
- main.c, lcd.c, and lcd.h

SESSION 6

- full instruction table
- DDRAM and CGRAM depth
- custom characters
- busy-flag reads
- complete reusable driver design

LCD command and character data

RS = 0

- byte is a command
- examples: clear, mode, cursor address
- EN pulse tells LCD to capture

RS = 1

- byte is character data
- ASCII code becomes a visible character
- EN pulse captures the data

R/W is tied to GND on the keypad shield, so this bridge writes only.

Four-bit transfer and C file roles

ONE BYTE ON FOUR DATA WIRES

- high nibble on D4-D7
- pulse EN
- low nibble on D4-D7
- pulse EN

SOURCE FILE RESPONSIBILITIES

- lcd.h declares functions
- lcd.c implements GPIO and timing
- main.c calls lcd_init and lcd_puts

Expected result

lcd_lab

1. Build native C
2. Upload to Uno
3. Attach the LCD Keypad Shield
4. Read two display lines

EXPECTED

Line 1: CE-LAB 2025. Line 2: AVR C only.

Pedestrian crossing scenario

Vehicles normally see green and pedestrians see red. One valid button press requests one safe crossing sequence.

Vehicle

red, yellow, green LEDs

Pedestrian

red and green LEDs

Input

active-low pushbutton

School-Zone Pedestrian Crossing Controller

A school wants to install a simple pedestrian crossing controller. An ATmega328P controls traffic signals for vehicles and pedestrians.

Under normal conditions, vehicles are allowed to move and pedestrians must wait. A pedestrian can press a crossing-request button. When a request is detected, the controller must safely stop vehicle traffic, allow pedestrians to cross for a limited time, warn them that crossing time is ending, and then return vehicle traffic to normal.

For this exercise, one pushbutton represents the pedestrian request sensor. Five LEDs represent the traffic signals:
Vehicle signals: Red, Yellow, Green
Pedestrian signals: Red, Green

Immediately after reset, the system must enter its normal state: vehicle Green ON, vehicle Yellow and Red OFF, pedestrian Red ON, pedestrian Green OFF.

School-Zone Pedestrian Crossing Controller

When the pedestrian button is pressed, the controller must perform the following sequence:

1. Keep the normal state for 1 second before changing the traffic signal.
2. Turn vehicle Green OFF and vehicle Yellow ON for 2 seconds.
3. Turn vehicle Yellow OFF and vehicle Red ON.
4. Wait 1 second before allowing pedestrians to cross.
5. Turn pedestrian Red OFF and pedestrian Green ON for 5 seconds.
6. Before ending the crossing period, make pedestrian Green blink three times, using 500 ms ON and 500 ms OFF.
7. Turn pedestrian Green OFF and pedestrian Red ON.
8. Keep vehicle Red ON for another 1 second.
9. Turn vehicle Red OFF and vehicle Green ON, returning to the normal state.

The same physical button must not accidentally start another complete crossing cycle simply because the pedestrian continues holding it. A new request should only be accepted after the previous cycle has finished and the button has been released.

One long press is one event



Without wait-for-release, the main loop sees the same held level as a new request.

Core exit ticket

1

Why can a build succeed while hardware behavior is wrong?

Answer: build checks translation, not every runtime assumption

2

Which file maps PORTB to the ATmega328P address?

Answer: avr/iom328p.h through avr/io.h

3

When should git revert be preferred over git restore?

Answer: when undoing a committed change while preserving history

4

What stops one held press from retriggering?

Answer: wait until the active-low button is released

Five ideas to remember

01 PlatformIO can build native AVR C without the Arduino framework.

02 Build, upload, runtime, and hardware are separate checkpoints.

03 avr-libc headers connect register names to device addresses and bit positions.

04 Git preserves tested states and provides safe recovery.

05 AI output needs datasheet, build, hardware, and requirement verification.

First Time Setup

```
git –version

git config --global user.name "Your Name"
git config --global user.email "your@email.com"
```

Configure your identity once

Git initialization and .gitignore

```
git init
git branch -M main
git status

.pio/
.vscode/c_cpp_properties.json
.vscode/launch.json
```

Why ignore generated files

- build output is reproducible
- machine-specific editor files create noise
- source and configuration remain reviewable

First working version

```
pio run
pio run -t upload
# observe D13 blink 500ms

git add .
git status
git commit -m "Blink 500"

git log --oneline
```

A known-good tag needs

- clean build
- successful upload
- expected physical observation

Status, diff, and staged diff

BEFORE git add

- git status
- git diff
- working-tree changes are visible
- index still holds previous snapshot

AFTER git add

- git status
- git diff --staged
- index contains proposed commit
- working tree may still have later edits

The staging area lets the engineer review exactly what will become history.

Second modification and commit

```
# edit main.c delay to 1000 ms
git status
git diff
pio run
git add .
git commit -m "Blink 1000"

git log --oneline
```

Behavior

- D2 / PD2 uses internal pull-up
- released HIGH
- pressed LOW
- button changes blink rate

Third modification and commit

```
# edit main.c delay to 250 ms
git status
git diff
pio run
git add .
git commit -m "Blink 250"

git log --oneline
```

Behavior

- D7 / PD7 becomes an output
- button LOW lights D7
- D13 heartbeat continues
- tag after both outputs are verified

Restoring an uncommitted file

```
# make one incorrect local edit
git status
git diff
git restore .
git status
```

Effect

- working file returns to the index version
- no commit is created
- discarded change is not preserved by Git

Inspecting a previous working commit

```
git log --oneline
git switch --detach [repo]
pio run
# optionally upload and test
```

Detached HEAD

- temporarily points at an exact commit
- main branch remains unchanged
- appropriate for inspection and testing
- do not begin normal work without a new branch

Returning to the latest version

```
git switch main
git status
git log --oneline
pio run
```

Verification

- HEAD attaches to main
- working tree matches latest commit
- latest behavior builds again

Reverting a committed fault

```
# commit version_4_committed_bug.c
git add .
git commit -m "Blink 2500"
pio run

git revert HEAD
git log --oneline
```

Result

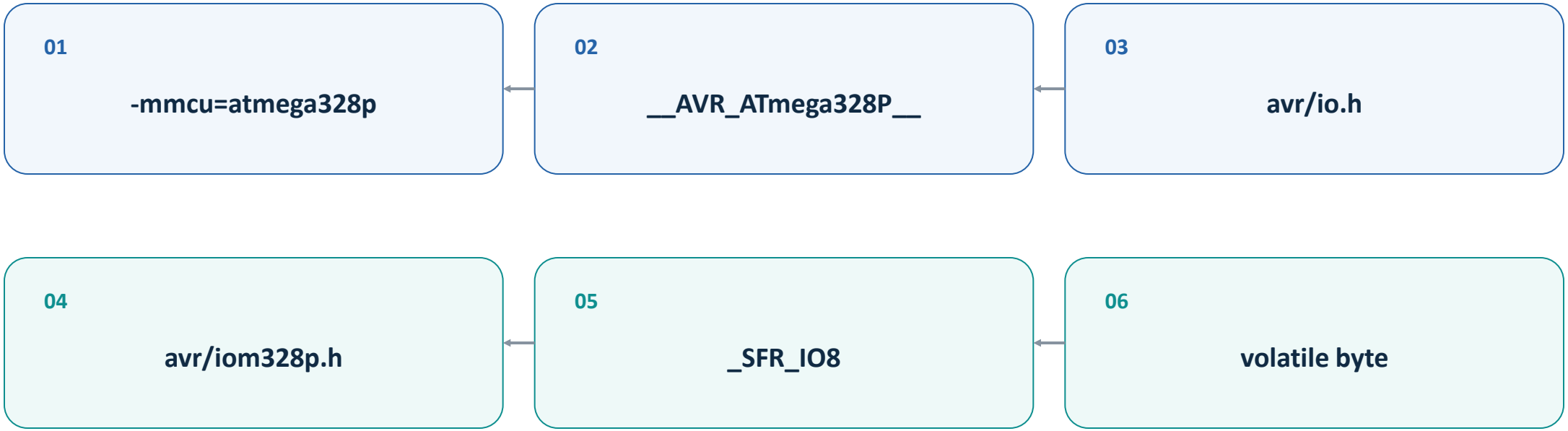
- faulty commit remains visible
- new commit reverses its patch
- shared history stays understandable
- rebuild and retest the restored behavior

Undo command decision rule

Situation	Preferred action	Reason
Uncommitted edit in one tracked file	git restore file	Return that file to the staged version
Committed fault in shared history	git revert commit	Add a visible inverse commit
Temporary test of older code	git switch --detach commit	Inspect without moving main
Exact branch rewrite with accepted data loss	git reset --hard commit	Destructive and rarely appropriate for beginners

Resolve the exact target before any destructive command. Never use reset --hard as a routine fix.

Header expansion from include to address



PORTB |= _BV(PORTB2) is a readable hardware access assembled from macros.

ELF, HEX, and memory use

ELF

- machine code plus sections
- symbols and debug information
- input to avr-size and disassembly
- richer than the programming image

INTEL HEX

- text records with address, type, data, checksum
- compact programming representation
- not source code and not executed as text
- created from ELF by avr-objcopy

Verified blink build: 164 bytes of application Flash and 0 bytes of static SRAM.

Pedestrian coding workshop

pedestrian_crossing_exercise

1. flowchart and pin table
2. outputs and safe sequence
3. long-press test
4. Git commit and evidence note

EXPECTED

One held press runs exactly one complete crossing sequence and returns to the safe idle state.